HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF INFORMATION & COMMUNICATION TECHNOLOGY

Comparative Evaluation of Traditional Machine Learning and Transformer-Based Models for Multi-label Code Comment Classification

A Comparative Study on the NLBSE'23 Dataset

Team Members

Nguyễn Khắc Thái Bình	20251324M
Lã Minh Trung	20251319M
Nguyễn Thu Uyên	20252279M
Nguyễn Thị Nhã Linh	20261262M
Nguyễn Đức Anh	20252469M

Course: Seminar 2

Instructor: Dr. Phuong Nguyen

University of L'Aquila, Italy

Academic Year: 2026–2027

Research Report

5 November 2026

Abstract

We compare five traditional machine-learning methods and seventeen RoBERTa/CodeBERT configurations for multi-label classification of 6,738 NLBSE'23 comment sentences in Python, Java, and Pharo. Our pipeline combines language-specific binary targets, exact-text-grouped partitions, word/character TF-IDF baselines, and pretrained neural encoders. Logistic regression is competitive with RoBERTa BCE on Java screening micro-F1 (0.5972 versus 0.5968), while the Transformer reference has larger advantages on Python and Pharo. CodeBERT with R-Drop leads neural screening on Python and Java (0.6479 and 0.6225); RoBERTa with focal loss leads Pharo (0.7152). The highest observed final-test micro-F1 values are 0.6541, 0.7034, and 0.7371 for Python, Java, and Pharo, respectively. Label-level analysis shows that higher aggregate F1 can coexist with weaker minority-label detection. Exploratory data analysis and stored learning curves explain the support imbalance and the role of validation-based stopping. We also provide interchangeable dataset storage adapters and reproducible result-generation scripts. Results are interpreted under the evaluation design and limitations stated in Section 8.

Keywords: code comments; multi-label classification; TF-IDF; RoBERTa; CodeBERT; class imbalance.

Contents

1 Introduction **1**

1.1 Research questions 1

1.2 Contributions and scope 1

2 Related Work and Background **2**

2.1 Code comment classification 2

2.2 Multi-label text classification 2

2.3 Traditional machine learning for software text 2

2.4 Pretrained Transformers for software engineering 2

2.5 Loss functions, regularization and evaluation 3

2.6 Positioning of the present study 3

3 Dataset, Problem Formulation and Exploratory Analysis **3**

3.1 NLBSE'23 dataset 3

3.2 Label taxonomy 4

3.3 Multi-label classification formulation 5

3.4 Class imbalance and label cardinality 5

3.5 Data preprocessing and duplicate grouping 5

3.6 Data splitting strategy 5

3.7 Exploratory data analysis 6

4 Software Architecture and Implementation **6**

4.1 System architecture 6

4.2 Data access and persistence 8

4.3 Preprocessing components 8

4.4 Feature extraction . . . 8
4.5 Model interfaces . . . 8
4.6 Training pipeline . . . 9
4.7 Evaluation pipeline . . . 9
4.8 Execution environment . . . 9

5 Methodology **9**
5.1 Traditional machine learning . . . 9
5.2 Transformer-based models . . . 10
5.3 Loss functions . . . 11
5.4 Experimental protocol . . . 12
5.4.1 Stage 1: grouped five-fold screening . . . 12
5.4.2 Stage 2: fixed approximately 80/10/10 evaluation . . . 12
5.5 Evaluation metrics . . . 13

6 Experimental Results **14**
6.1 Traditional ML versus the Transformer reference: RQ1 . . . 14
6.2 Transformer configuration comparison: RQ2 . . . 15
6.3 Final evaluation . . . 15
6.4 Cross-validation versus test ranking . . . 17
6.5 Per-label performance: RQ3 . . . 17
6.6 Learning dynamics and early stopping . . . 19
6.7 Illustrative comments and quantitative difficult-label analysis . . . 19
6.8 Published benchmark context . . . 21

7 Discussion **21**
7.1 When traditional ML is competitive . . . 21
7.2 Backbones, objectives, and interactions . . . 22
7.3 Aggregate and rare-label performance . . . 22
7.4 Practical implications and generalization . . . 22

8 Limitations and Threats to Validity **22**
8.1 Selection and statistical validity . . . 22
8.2 Data and external validity . . . 23
8.3 Metrics and experimental coverage . . . 23
8.4 Artifact and software scope . . . 23

9 Reproducibility **23**
9.1 Source, data and environment . . . 23
9.2 Installation and dataset preparation . . . 24

9.3 Training and evaluation 24
9.4 Artifacts, visualizations and software checks 24
9.5 Report build 25

10 Conclusion **25**

A Complete Screening Metrics **28**

B Pharo Reference Per-label Results **30**

C Section-to-Artifact Map **30**

List of Figures

1 Positive label prevalence calculated from the actual sentence-level targets. Bars compare labels within each language; counts are provided in Table 3. 6
2 Sentence-length ECDFs using words and characters; horizontal axes are logarithmic. Every step is computed from the checked-in normalized text. Word counts are not tokenizer counts. 7
3 Conditional positive-label co-occurrence in the full descriptive population. Diagonal entries equal one; graph statistics used for training are estimated separately within the training split. Long axis labels are abbreviated only in this figure. 7
4 Software modules with interchangeable staged-dataset stores. FileStore and MemoryStore implement the same byte interface before the shared loader validates records and creates a StageBundle. 7
5 Experimental flow from category-wise records to model comparison. Configurations, metrics, plots and manifests are exported through filesystem run directories. 8
6 Reference classification paths. Neural sigmoid outputs make separate binary label decisions; the TF-IDF branch uses estimator-specific hard predictions. Attention and graph variants modify the reference neural head. 10
7 Actual two-stage protocol. Full-population screening and within-fold checkpoint selection limit independence. The later 10% test split is not an untouched outer test set. 13
8 Five traditional methods versus RoBERTa E1 on Stage 1 micro-F1. Error bars are sample SD across five folds, not confidence intervals. LR denotes logistic regression. 14
9 Stage 1 means and Stage 2 test micro-F1 for the five evaluated Transformers. Scores are not independent replications and differences must not be read as generalization-gap estimates. 17
10 Python test F1 by label and final Transformer configuration. Every value is read from stored per-label metrics; supports are given in Table 12. 18
11 Java test F1 by label and configuration. High Ownership scores are based on only eleven positive test assignments. 18
12 Pharo test F1 by label and configuration. E1 leads aggregate test micro-F1, while some of its individual label scores are surpassed. 19
13 Stored logistic-regression refit diagnostics. Top: training Hamming loss; bottom: validation F1. The horizontal axis is the maximum solver iteration count on a log scale, not a neural epoch. 20

14 Saved neural histories for E1 and E15. Stars mark selected validation epochs. Loss values are objective-specific; the bottom row compares the common validation micro-F1 metric. . 20

List of Tables

1 Related studies and their relationship to the present comparison. 3
2 Verified sentence and target counts. Groups identify exact whitespace-normalized text. . . 4
3 Positive support by label and Stage 2 split; prevalence uses the full language population. . 4
4 Software components and verification scope. 9
5 Verified traditional model settings from the builders. All use the same word/character TF-IDF construction. 10
6 Verified E1–E17 configurations. R: RoBERTa; C: CodeBERT. Unspecified settings inherit the common configuration. 11
7 Stage 1 micro-F1: mean and sample standard deviation across five folds. E1 uses fold validation for checkpoint selection. 14
8 Transformer screening mean and sample SD. E4 shares E1 scores in all three languages and is omitted because threshold tuning is deferred. All 51 run records remain available. . 15
9 Python Stage 2 test results. HL is Hamming loss (lower is better). 16
10 Java Stage 2 test results. HL is Hamming loss (lower is better). 16
11 Pharo Stage 2 test results. HL is Hamming loss (lower is better). 16
12 Python test label performance. The highest observed test micro-F1 Transformer is E15; comparator choice is retrospective and descriptive. 17
13 Java test label performance. The highest observed test micro-F1 Transformer is E17; comparator choice is retrospective and descriptive. 18
14 Actual annotated sentences illustrating information types. These are dataset examples, not model errors or predictions. 21
15 Published NLBSE'23 results from the organizers' Table III; context only, not directly comparable with our grouped splits. 21
16 Python Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted. 28
17 Java Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted. 29
18 Pharo Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted. 29
19 Pharo test label performance. The highest observed test micro-F1 Transformer is E1; comparator choice is retrospective and descriptive. 30
20 Primary artifacts supporting each report component. 30

1 Introduction

Code comments complement source code by explaining intent, usage, design decisions, parameters, and implementation constraints. Their information is useful for documentation navigation, maintenance, and identifying the kinds of explanation available to a developer. A classifier can provide an information-type index over comment sentences, although classification performance alone does not establish a downstream maintenance benefit.

The prediction unit in this study is a sentence within a class comment. A sentence may describe both what a component does and how to use it. Its annotation is therefore a set of categories, represented by several binary targets. Treating the task as mutually exclusive multi-class classification would discard legitimate co-occurring labels. It would also obscure the distinction between a sentence and the many sentence–category records used to distribute the original data.

The NLBSE'23 benchmark provides a concrete setting for studying comment information types across three programming languages [11]. Language-specific taxonomies, short fragments, identifier-like tokens, repeated wording, and uneven label supports complicate evaluation. A high score on frequent categories may conceal weak recall on rare ones. Sparse lexical models offer a relatively inexpensive reference, while pretrained encoders may capture contextual distinctions that TF-IDF cannot represent directly. Whether those advantages appear consistently must be measured under comparable data partitions.

This study addresses a bounded empirical gap: how the implemented lexical baselines and neural configurations compare on the repository's shared partitions, and whether aggregate gains extend across labels. It does not claim that such comparisons are absent from the wider literature. The available runs cannot establish superiority on new projects, isolate every architectural factor, or provide a new official competition ranking.

1.1 Research questions

RQ1.
How do traditional machine-learning approaches compare with a reference Transformer model for multi-label code comment classification? The comparison uses shared Stage 1 five-fold partitions and RoBERTa E1 as the reference.

RQ2.
Which Transformer configurations achieve the strongest performance across Python, Java, and Pharo? All E1–E17 screening results are compared, followed by the available, incomplete final-evaluation subset.

RQ3.
Do improvements in aggregate classification performance correspond to improvements across individual labels? Per-label precision, recall, F1, and support are examined alongside micro- and macro-F1.

1.2 Contributions and scope

We provide a shared sentence-level formulation and grouped pipeline; compare five lexical learners with seventeen neural configurations; analyze aggregate, label-level and learning-dynamics results; and supply reproducible software with interchangeable dataset stores. Figures and tables use the team's recorded experiments. Revision-time software verification is distinguished from historical experiments in Section 9; source provenance is collected in Appendix C.

2 Related Work and Background

2.1 Code comment classification

Haouari et al. [5] studied Java commenting practices and developed a taxonomy of the information conveyed by comments. This distinguishes understanding a comment's role from merely locating comment text. Pascarella and Bacchelli [12] investigated six Java open-source systems, manually classified more than 2,000 comments, and evaluated automatic line-level classification. Their work motivates information-type classification while also showing that taxonomies and annotation units are part of the task definition.

Language conventions matter. Rani et al. [15] examined Pharo comment evolution, information types and adherence to class-comment templates. Their Java/Python study [13] found that conventions do not adequately specify all information needs. These findings motivate our language-specific label spaces instead of assuming that one taxonomy transfers unchanged across ecosystems.

Information types are distinct from comment quality. Rani et al.'s systematic review [14] analyzed 47 relevant studies and identified a strong emphasis on Java and comment–code consistency. Predicting Usage or DevelopmentNotes does not establish correctness, completeness or freshness. Our target is agreement with annotated information categories, not a direct quality or productivity measure.

NLBSE'23 provides a shared dataset and binary baseline classifiers. Its overview [6] reports submitted models and official evaluation results. STACC [1] uses SentenceTransformers-based binary classifiers, providing a concrete alternative to the token encoder and linear head used by our reference model. Published benchmark scores are presented separately in Section 6.8, since official partitions and aggregation differ from our grouped protocol.

2.2 Multi-label text classification

Zhang and Zhou [19] distinguish problem transformations from algorithms designed for label sets. Binary Relevance fits one binary learner per label. Classifier Chains condition later classifiers on preceding labels [16], allowing dependencies but introducing order sensitivity and error propagation. Our chain uses one seeded random order. A neural sigmoid head also produces independent binary outputs, while its shared encoder learns common features. Label attention and graph variants add explicit label-related representations.

2.3 Traditional machine learning for software text

Strong lexical baselines are necessary for short text and modest training sets. Wang and Manning [18] show that text feature choices and classifier variants materially affect baseline comparisons. Their sentiment/topic tasks differ from code comments, but motivate our inclusion of word bigrams and multiple linear learners. Character n-grams additionally match subword and identifier-like patterns. Logistic regression, SVM, SGD and Random Forest test whether contextual encoders improve over that shared sparse representation. Class weighting changes fitting costs; feature design changes the information available to the learner.

2.4 Pretrained Transformers for software engineering

BERT introduced bidirectional Transformer pretraining and task-specific fine-tuning [3]. RoBERTa revisited its pretraining and optimization recipe [9]; CodeBERT learns from programming and natural language with a hybrid objective [4]. Their pretraining backgrounds motivate a comparison but do not guarantee a language-independent winner. Our models receive comment text without surrounding code or syntax

trees. STACC illustrates another design point based on sentence representations and binary heads [1]; it is discussed as related work rather than claimed as a reproduced experiment.

2.5 Loss functions, regularization and evaluation

Focal loss suppresses easy decisions [8], whereas ASL handles positive and negative assignments differently [2]. We compare them with ordinary and positive-weighted BCE. R-Drop regularizes agreement across dropout realizations [7]; AdamW decouples weight decay from adaptive optimization [10]. Weighting, consistency and optimization address different mechanisms and should not be treated as interchangeable imbalance remedies.

Saito and Rehmsmeier [17] explain why precision–recall evaluation is informative under class imbalance. We therefore use AP, micro/macro-F1 and label supports instead of relying on accuracy. Selection and statistical limitations are consolidated in Section 8.

2.6 Positioning of the present study

Our contribution combines shared exact-text-grouped partitions, five lexical methods, seventeen neural configurations and label-level analysis. It is a comparative study, not a new taxonomy or architecture. Table 1 distinguishes the scope of the most relevant comment studies.

Table 1: Related studies and their relationship to the present comparison.

Study	Verified focus	Relationship / boundary
Haouari et al. [5]	Java commenting practices and taxonomy	Information-type distinctions
Pascarella & Bacchelli [12]	Six Java systems; manual/automatic classification	Earlier classification evidence; different unit/taxonomy
Rani et al. [15]	Pharo comment evolution and templates	Language-specific information types
Rani et al. [13]	Java/Python conventions	Guidelines versus actual comment content
Rani et al. [14]	Comment-quality literature review	Categories are distinct from quality
NLBSE'23 [6]	Competition and official RF baselines	Shared benchmark; different evaluation split
STACC [1]	SentenceTransformers binary classifiers	Published neural comparator; not rerun here
Present study	Three languages; TF-IDF and E1–E17	Shared grouped screening and per-label comparison

3 Dataset, Problem Formulation and Exploratory Analysis

3.1 NLBSE'23 dataset

The dataset has 2,555 Python, 2,418 Java, and 1,765 Pharo sentences, totaling 6,738. The checked-in raw files are `data/python.csv`, `data/java.csv`, and `data/pharo.csv`. Each contains repeated sentence IDs: one record per category, including negative assignments. Thus Python has 12,775 raw records, Java 16,926, and Pharo 12,355; these are not counts of independent sentences. The dataset configuration pins an upstream source commit and download URLs.

We pivoted the raw targets by sentence ID and category and compared every resulting target against `data/k_fold_data/<language>/all.csv`. It also checked that the final splits reconstruct the same

sentence population and targets. Counts in Table 2 are computed from these files rather than copied from manuscript prose.

Table 2: Verified sentence and target counts. Groups identify exact whitespace-normalized text.

Language	N	L	Positive	Negative	Cardinality	Groups
Python	2555	5	2864	9911	1.121	2299
Java	2418	7	2845	14081	1.177	1668
Pharo	1765	7	2043	10312	1.158	1620

3.2 Label taxonomy

Label order is taken from each language's `label_cols.json`; original capitalization and spelling are retained. Python uses DevelopmentNotes, Expand, Parameters, Summary, and Usage. Java uses Expand, Ownership, Pointer, deprecation, rational, summary, and usage. Pharo uses Classreferences, Collaborators, Example, Intent, Keyimplementationpoints, Keymessages, and Responsibilities. There are 19 language-specific categories across the corpora, while each model run predicts only its language's five or seven labels.

For interpretation, Summary/summary and Intent describe purpose; Usage/usage and Example concern use; Parameters concern input descriptions; DevelopmentNotes concern development or maintenance information; Ownership and deprecation identify attribution and obsolete functionality. Expand and rational concern elaboration or rationale; Pointer and Classreferences identify references; Pharo's remaining names concern interactions, responsibilities, messages, and implementation details. These are descriptive glosses of the category names, not a newly verified annotation manual. No complete operational annotation guide or inter-annotator agreement record was found locally, so fine semantic boundaries should not be inferred from these glosses.

Table 3: Positive support by label and Stage 2 split; prevalence uses the full language population.

Language	Label	Total +	Prevalence (%)	Train +	Val +	Test +
Python	DevelopmentNotes	312	12.21	250	31	31
Python	Expand	504	19.73	403	50	51
Python	Parameters	794	31.08	635	79	80
Python	Summary	454	17.77	364	45	45
Python	Usage	800	31.31	640	80	80
Java	Expand	632	26.14	506	63	63
Java	Ownership	115	4.76	95	9	11
Java	Pointer	364	15.05	294	35	35
Java	deprecation	127	5.25	102	13	12
Java	rational	280	11.58	224	28	28
Java	summary	415	17.16	332	41	42
Java	usage	912	37.72	730	91	91
Pharo	Classreferences	77	4.36	62	8	7
Pharo	Collaborators	127	7.20	102	12	13
Pharo	Example	748	42.38	599	74	75
Pharo	Intent	218	12.35	175	22	21
Pharo	Keyimplementationpoints	232	13.14	186	23	23
Pharo	Keymessages	305	17.28	243	31	31
Pharo	Responsibilities	336	19.04	269	33	34

3.3 Multi-label classification formulation

For a language-specific dataset $D = \{(x_i, \mathbf{y}_i)\}_{i=1}^{N}$, each comment x_i has a binary target vector

$$\mathbf{y}_i \in \{0,1\}^L, \qquad L \in \{5,7\}. \tag{1}$$

Here $y_{i\ell} = 1$ denotes a positive assignment to label ℓ; $y_{i\ell} = 0$ denotes a negative assignment to that category, not a negative sentiment or an unusable sentence. Multi-class classification would choose a single class from L mutually exclusive possibilities. Multi-label classification instead predicts a subset, including multiple positives when appropriate.

In the reference neural formulation, an encoder supplies a sentence representation h_i, a linear head produces $z_{i\ell}$, and

$$p_{i\ell} = \sigma(z_{i\ell}), \qquad \widehat{y}_{i\ell} = \mathbb{1}[p_{i\ell} \geq \tau_\ell]. \tag{2}$$

Each output is a separate Bernoulli decision; probabilities need not sum to one. This factorized output does not imply that labels or shared hidden features are empirically independent. Chains and graph variants explicitly introduce additional dependency mechanisms.

3.4 Class imbalance and label cardinality

Label cardinality is $N^{-1} \sum_{i,\ell} y_{i\ell}$; label density divides cardinality by L. Cardinalities are 1.121, 1.177, and 1.158 for Python, Java, and Pharo, respectively. Their densities are 22.42%, 16.81%, and 16.54%. A typical sentence has slightly more than one positive label and several negatives, explaining the excess of negative assignments. Across the three corpora there are 7,752 positives and 34,304 negatives, over 42,056 binary decisions.

Python, Java, and Pharo contain 284, 397, and 231 sentences with more than one positive category. None has an all-zero target in this population. Within-language imbalance is also substantial: Pharo Classreferences has 77 positives compared with 748 for Example; Java Ownership has 115 compared with 912 for usage. Final-test rare-label supports can be very small (seven Classreferences and eleven Ownership positives), limiting the precision of label-level conclusions.

3.5 Data preprocessing and duplicate grouping

`raw_to_rows` validates IDs, class/text consistency, binary targets, complete categories, and the raw partition values. It pivots into one sentence record and normalizes whitespace with `" ".join(text.split())`. Text equality then determines `group_id`. It does not deduplicate away rows: different sentence IDs with the same normalized text remain together as one group. The numbers of distinct text groups are 2,299, 1,668, and 1,620, leaving 256, 750, and 145 rows beyond one row per group. These excess counts are not numbers of duplicate pairs.

This stage performs no additional stemming or identifier replacement. The TF-IDF vectorizers separately use their default lowercasing, whereas the tokenizer receives the normalized string. Earlier preprocessing in the upstream dataset must not be confused with new preprocessing performed here. Exact grouping does not detect paraphrases, punctuation variants, or all near duplicates.

3.6 Data splitting strategy

`grouped_parts` aggregates each text group's positive counts and size, then greedily assigns whole groups to parts. A seeded permutation and size ordering feed a cost that balances per-label positives and sample counts relative to targets. This is a custom heuristic, not a verified implementation of a named iterative-stratification algorithm. Stage 1 uses five parts over the complete population. Stage 2 creates ten parts and

combines eight for training, one for validation, and one for test. The inherited raw `partition` is validated but does not determine these new splits.

Final train/validation/test sizes are 2,044/255/256 (Python), 1,935/241/242 (Java), and 1,413/176/176 (Pharo). We verified zero exact-group overlap within each CV split and across final splits, and exactly one validation appearance per sentence across five folds. Class-level separation is not enforced: 97 of 119 Python test classes, 93 of 112 Java test classes, and 74 of 93 Pharo test classes also occur in training. The splits therefore do not evaluate unseen-class or unseen-project generalization.

3.7 Exploratory data analysis

We examine prevalence, sentence length and label co-occurrence before interpreting model results. Figure 1 makes the within-language imbalance visible: Python Usage and Parameters are much more frequent than DevelopmentNotes, while Java usage and Pharo Example dominate their respective corpora. A common threshold and micro-F1 can therefore favor frequent information types.

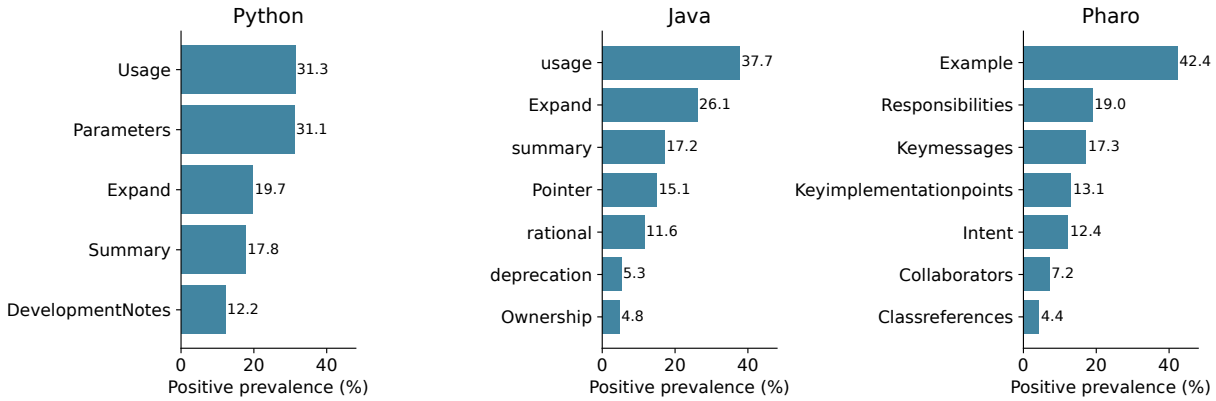


Figure 1: Positive label prevalence calculated from the actual sentence-level targets. Bars compare labels within each language; counts are provided in Table 3.

Figure 2 shows empirical cumulative distributions rather than fitted curves. Median whitespace-delimited lengths are 6 words for Python, 6 for Java and 7 for Pharo; the 95th percentiles are 13, 16 and 22. Java has a longer extreme tail (maximum 115 words), while the longest Python and Pharo entries have 26 and 47. These counts describe space-separated words, not pretrained subword tokens, so they do not by themselves measure truncation at 128 token positions. Short fragments help explain why lexical cues can remain competitive and why contextual information may be missing even for a powerful encoder.

Figure 3 reports conditional co-occurrence, $P(\ell_b = 1 \mid \ell_a = 1)$. It is generally asymmetric because conditioning uses the support of the row label. It motivates chains and label graphs, but association alone does not guarantee that modeling dependence improves prediction. For rare categories these conditional estimates can be sensitive to a few examples; our graph construction therefore uses only training indices rather than the full EDA matrix.

4 Software Architecture and Implementation

4.1 System architecture

Our application separates data preparation, feature/model construction, training, evaluation and visualization. Python entry points and Bash wrappers orchestrate experiments. Both model families consume shared validated staged datasets, while fitting and numerical scoring remain specialized. Figures 4 and 5 show the interactions.

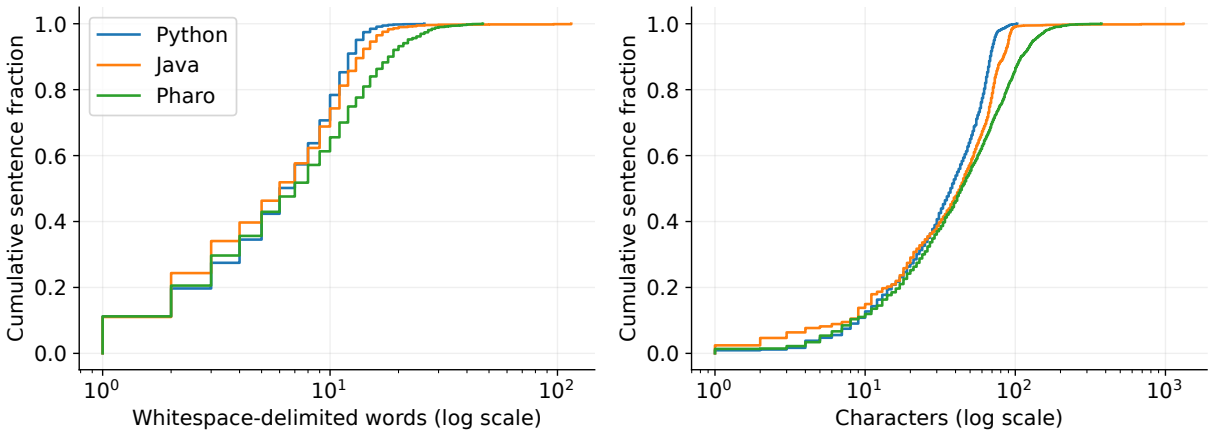


Figure 2: Sentence-length ECDFs using words and characters; horizontal axes are logarithmic. Every step is computed from the checked-in normalized text. Word counts are not tokenizer counts.

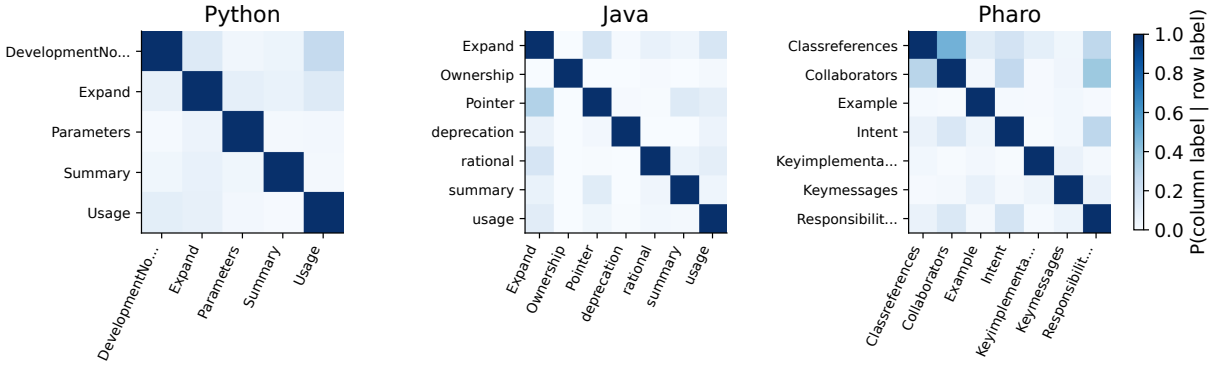


Figure 3: Conditional positive-label co-occurrence in the full descriptive population. Diagonal entries equal one; graph statistics used for training are estimated separately within the training split. Long axis labels are abbreviated only in this figure.

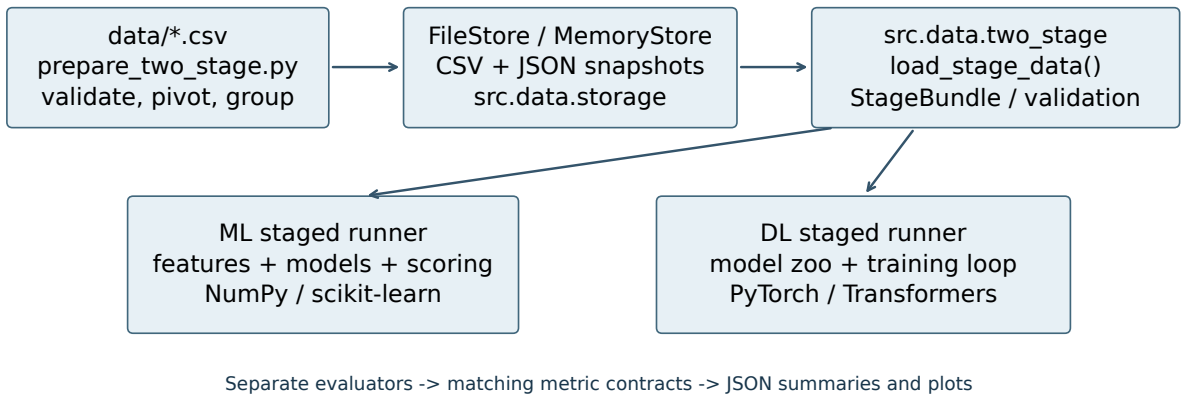


Figure 4: Software modules with interchangeable staged-dataset stores. FileStore and MemoryStore implement the same byte interface before the shared loader validates records and creates a StageBundle.

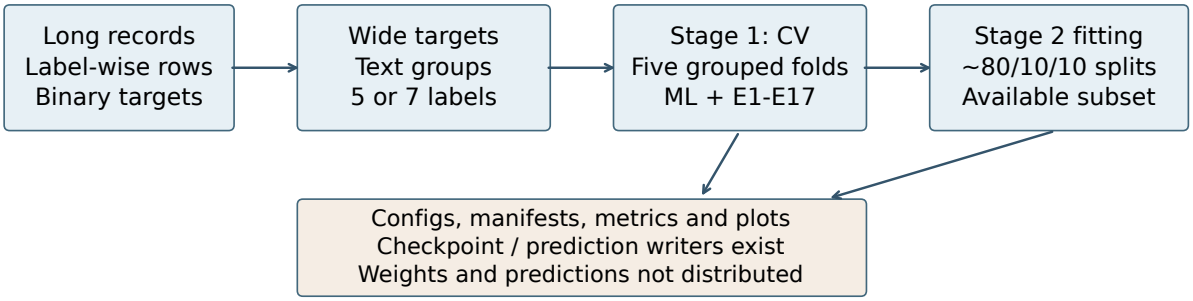


Figure 5: Experimental flow from category-wise records to model comparison. Configurations, metrics, plots and manifests are exported through filesystem run directories.

4.2 Data access and persistence

The `ByteStore` protocol provides `read_bytes(path)` and `write_bytes(path,value)`. `FileStore` persists bytes through filesystem paths. `MemoryStore` maintains an isolated byte dictionary and can import a prepared dataset snapshot. Both preserve path keys and raise `FileNotFoundError` for missing data. Memory writes change only the snapshot; file writes change the target file.

The staged loader accepts an injected store. CSV parsing, label validation, group checks, masks and fingerprints remain independent of its backend. Both ML and neural CLIs expose `--storage file|memory`: only store construction changes. After the initial memory snapshot, loader reads and manifest copying use the injected store; feature extraction, fitting and evaluation are unchanged. Six passing tests cover read-/write behavior, snapshot isolation, shared validation failures and equivalence of all six real language–stage datasets, including masks, folds and fingerprints.

The adapter layer was added during report revision; archived scientific results predate it and were not retrained. Its scope is staged-dataset persistence. Run directories, plots, tokenizers and checkpoint exports still have explicit filesystem output paths. This allows dataset access to switch independently while keeping shareable result artifacts. A fully interchangeable artifact-export layer is a separate extension.

4.3 Preprocessing components

`prepare_two_stage.py` validates raw records, pivots targets, normalizes text, groups duplicates and constructs partitions. `two_stage.py` validates saved partitions before fitting. `StageBundle` carries texts, targets, masks, indices and provenance. A population fingerprint identifies canonical records and label order; a file fingerprint identifies dataset bytes.

4.4 Feature extraction

The ML branch fits a fresh word/character TF-IDF `FeatureUnion` inside each training fold. The neural branch uses `TokenizedNLBSEDataset` to return token IDs, attention masks, binary targets and label masks, padded/truncated to 128 positions. They share target/split semantics without sharing tokenization.

4.5 Model interfaces

Classical estimators expose scikit-learn's `fit`, `predict`, and probability or decision-score methods. `predict_scores` supplies ranking scores for AP. Neural forward calls return logits and optional masked

loss through `SequenceClassifierOutput`. The `MODEL_ZOO` registry maps E1–E17 to configurations and factories. Positive weights and label graphs use current training indices only.

4.6 Training pipeline

Classical fitting clones a model and learns its vectorizer on training text; final validation is diagnostic. Separate refits at iteration/tree limits produce ML curve files. Neural training applies AdamW, warmup/-linear decay, clipping, epoch validation, early stopping and best-state restoration. Stored histories support Section 6.6.

4.7 Evaluation pipeline

Both branches return a common schema: micro/macro precision, recall and F1, Hamming loss, AP and label counts. ML uses NumPy/scikit-learn; neural evaluation accumulates masked tensor counts. Summaries use fold means and sample SD. JSON metrics and manifests support reproducible figure/table generation; artifact-availability limits are stated in Section 8.

4.8 Execution environment

CPU/CUDA selection is supported. Dependencies include PyTorch, Transformers, scikit-learn, NumPy, pandas, PyYAML, matplotlib, tensorboard, tqdm and joblib. Section 9 gives version ranges and commands. Three Jupyter EDA notebooks have saved outputs; a dedicated Colab setup has not been validated. Storage tests and report builds use Python 3.12.10 on Windows, which identifies the revision environment rather than the original training machine.

Table 4: Software components and verification scope.

Component	Responsibility	Verification
FileStore / MemoryStore	Interchangeable dataset byte access	Six passing contract/integration tests
Shared loader	Schema, groups, coverage, masks and hashes	Six real dataset snapshots equivalent
TF-IDF and ML builders	Five learners plus diagnostic dummy	Code/configurations and stored results
Model zoo / neural training	E1–E17 and checkpoint policies	Code/configurations and histories
Evaluation / visualization	Common metrics, EDA and figures	Numerical checks and document QA
Filesystem exports	Metrics, plots, tokenizer/model writes	Code inspected; archived coverage varies
Jupyter EDA	Dataset exploration	Three notebooks with saved outputs

5 Methodology

5.1 Traditional machine learning

For term t and document x, TF-IDF multiplies a term count by inverse document frequency. With the vectorizer defaults, smoothed IDF is $1+\log[(1+n)/(1+\mathrm{df}(t))]$; feature vectors are normalized within each vectorizer. Word unigrams/bigrams and `char_wb` character 3–5-grams are fit separately, then concatenated by `FeatureUnion`. Both have `min_df=2` and at most 20,000 features, giving an upper bound of 40,000 concatenated dimensions, not a guaranteed vocabulary size. The defaults include lowercasing and no explicit stop-word list. No pretrained sentence embedding is used by these baselines.

Logistic regression minimizes a regularized logistic objective and supplies probability estimates. Linear SVM learns margin-based separators, with raw decision scores for AP rather than calibrated probabilities. SGD uses a stochastic logistic-loss optimizer, not a different feature representation. Random Forest aggregates decision trees. One-vs-Rest wraps each of these estimators to produce binary decisions per label. Classifier Chain instead uses logistic estimators in a random, seed-controlled label order and conditions later estimators on preceding labels. Table 5 gives explicit non-default builder settings; other scikit-learn defaults depend on the installed version.

Table 5: Verified traditional model settings from the builders. All use the same word/character TF-IDF construction.

Method	Explicit settings
Logistic regression	One-vs-Rest; $C = 5$; balanced class weights; 2,000 iterations maximum
Classifier Chain	Logistic base with $C = 5$, balanced weights, 2,000 iterations; random order; seed 42
Linear SVM	One-vs-Rest LinearSVC; $C = 1$; balanced class weights
SGD	One-vs-Rest; logistic loss; 2,000 iterations; tolerance 10^{-3}; balanced weights; seed 42
Random Forest	One-vs-Rest; 300 trees; balanced weights; seed 42; all CPU jobs
Majority dummy	One-vs-Rest most-frequent target; diagnostic baseline

The majority dummy is an additional diagnostic, not a sixth substantive learned method. Its five-fold records are included in the audit but no final dummy record is present. Hard predictions use each estimator's own `predict` method; no common neural sigmoid threshold is imposed on SVM margins.

5.2 Transformer-based models

RoBERTa and CodeBERT checkpoints are loaded through Hugging Face Transformers. The baseline model uses the first-token representation from `RobertaModel`, with hidden dimension 768, followed by a single linear $768 \to L$ head. It does not use the full default `RobertaForSequenceClassification` head. The tokenizer returns padded/truncated sequences of length 128 and attention masks. Heads and loss masks are sized for one programming language at a time. Figure 6 shows this reference path and the alternative feature path.

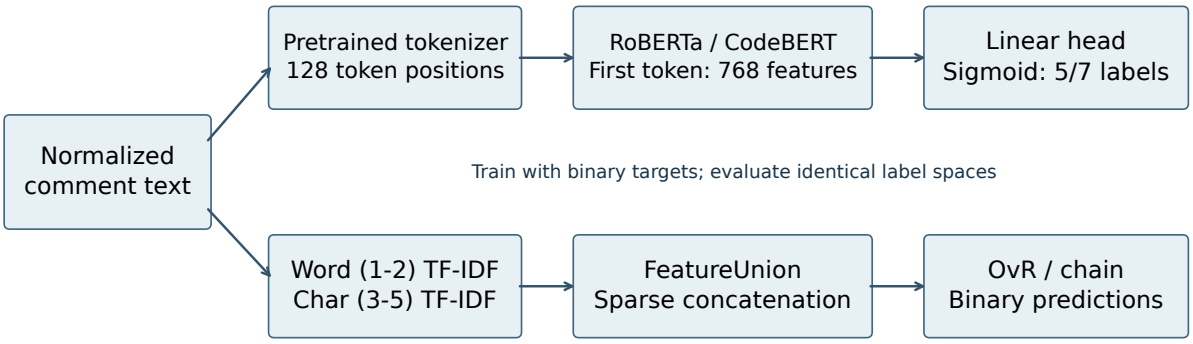


Figure 6: Reference classification paths. Neural sigmoid outputs make separate binary label decisions; the TF-IDF branch uses estimator-specific hard predictions. Attention and graph variants modify the reference neural head.

Table 6 lists all verified configurations. Backbone changes are represented by E11 and its CodeBERT derivatives. Loss variants include E2/E3/E7/E17. E5/E6/E9/E12 modify label representation or alignment; E8/E10/E15/E16 add regularization; E13/E14/E16 change optimization. These groups overlap, and E16 deliberately combines two interventions.

Table 6: Verified E1–E17 configurations. R: RoBERTa; C: CodeBERT. Unspecified settings inherit the common configuration.

ID	Backbone	Intervention and explicit settings
E1	R	BCE; first-token linear head
E2	R	Positive-weighted BCE; training negatives/positives per label
E3	R	Binary focal loss; $\gamma = 2$, no alpha weighting
E4	R	BCE; final-validation threshold grid 0.05–0.95 in steps of 0.05; deferred during CV
E5	R	Learned label queries attending over non-padding tokens
E6	R	E5 plus sentence–label contrastive term; weight 0.1, temperature 0.1
E7	R	ASL; $\gamma_+ = 1$, $\gamma_- = 4$, negative clip 0.05; detached focusing factor
E8	R	BCE plus R-Drop; consistency coefficient 1
E9	R	Residual label GNN v2; graph threshold 0.1, hidden size 128; patience 3, minimum 5 epochs; AP tie-breaker
E10	R	Clean/adversarial embedding BCE; $\epsilon = 1$, adversarial weight 1
E11	C	BCE; reference CodeBERT backbone change
E12	R	Label attention followed by GNN; threshold 0.1, hidden size 128
E13	C	BCE; encoder LR 2×10^{-5}; head LR 10^{-4}
E14	C	E13 learning rates plus layer-wise decay 0.9
E15	C	BCE plus R-Drop; consistency coefficient 1
E16	C	E14 layer-wise learning rates plus R-Drop coefficient 1
E17	C	Binary focal loss; $\gamma = 2$, no alpha weighting

E5 computes a token-attention distribution for each learned label query and a label-specific context/logit. E6 additionally aligns projected first-token representations with positive label queries using a uniform multi-positive target. E9 estimates a cosine co-occurrence graph from training labels, adds self loops, normalizes by degree, and refines label embeddings through a two-layer graph residual. Its second graph layer is zero-initialized so the head begins at the ordinary linear-head scale. All fifteen stored E9 screening folds report head version 2. E12 applies graph message passing to attention-derived contexts. Graphs are per-language (5/7/7 nodes), not a joint 19-node experiment.

E10 obtains an embedding gradient, masks padding positions, and adds a per-sentence L2-normalized perturbation. Its loss averages clean and perturbed BCE with the configured weight. E14/E16 assign the final encoder layer the encoder LR and layer i the rate $\eta_e d^{K-1-i}$, with embedding rate $\eta_e d^K$, $d = 0.9$ and K encoder layers. Head LR is separate. Optimizer groups apply the same configured weight decay, including biases and normalization parameters; a standard no-decay grouping must not be assumed.

5.3 Loss functions

For M observed binary targets and sigmoid probabilities p, the reference objective is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{M} \sum_{i,\ell} \left[y_{i\ell} \log p_{i\ell} + (1 - y_{i\ell}) \log(1 - p_{i\ell}) \right]. \tag{3}$$

The implementation uses numerically stable BCE with logits and averages only selected mask entries. In the per-language staged runs, targets are fully observed within each selected split.

Weighted BCE replaces the positive term with $w_\ell y_{i\ell} \log p_{i\ell}$, where $w_\ell = n_\ell^- / n_\ell^+$ is calculated from current training targets only. Labels with no positives or no negatives receive neutral weight 1. This shifts the cost of positive errors and can trade precision for recall. It does not change the evaluation prevalence.

For $p_t = yp + (1 - y)(1 - p)$, the implemented focal objective is

$$\mathcal{L}_{\text{focal}} = -\frac{1}{M} \sum_{i,\ell} (1 - p_{t,i\ell})^\gamma \log p_{t,i\ell}, \qquad \gamma = 2. \tag{4}$$

As p_t approaches one, the focusing multiplier suppresses easy examples. With $\gamma = 0$, it reduces to unweighted BCE. Although the class supports optional alpha balancing, E3/E17 set alpha to null, so no explicit positive/negative alpha factor is applied. Focusing on hard decisions is distinct from automatically equalizing label supports.

ASL uses different positive and negative exponents and clips negative probabilities before the negative log term. Its configured focusing factor is detached from gradient computation. This is a particular implementation choice that should accompany any attempted reproduction.

For two dropout realizations with per-label Bernoulli distributions $P_\ell = \text{Bern}(p_\ell)$ and $Q_\ell = \text{Bern}(q_\ell)$, the R-Drop implementation uses

$$\mathcal{L}_{\text{RDrop}} = \tfrac{1}{2}(\mathcal{L}^{(1)}_{\text{BCE}} + \mathcal{L}^{(2)}_{\text{BCE}}) + \alpha\mathcal{R}, \tag{5}$$

$$\mathcal{R} = \frac{1}{2M}\sum_{i,\ell}\left[D_{\text{KL}}(P_{i\ell}\|Q_{i\ell}) + D_{\text{KL}}(Q_{i\ell}\|P_{i\ell})\right], \tag{6}$$

$$D_{\text{KL}}(P\|Q) = p\log\frac{p}{q} + (1-p)\log\frac{1-p}{1-q}, \qquad \alpha = 1. \tag{7}$$

Training performs two forward passes; evaluation performs one. The consistency term regularizes predictions across dropout realizations. It does not directly balance positive and negative assignments. Weighted BCE and ASL explicitly address asymmetric target contributions; R-Drop and adversarial perturbations primarily target robustness/regularization.

5.4 Experimental protocol

5.4.1 Stage 1: grouped five-fold screening

Every language's full sentence population is split into five text-grouped folds. Each traditional model fits a new vectorizer and classifier in each training fold. Each Transformer starts from its pretrained checkpoint with base seed 42 plus fold index (42–46). Mean fold micro-F1 is used to rank configurations; the recorded standard deviation is sample SD across five folds, not uncertainty across five independent dataset draws or a confidence interval.

Neural training evaluates the held-out fold after each epoch, selects the checkpoint with maximum fold micro-F1, and reports that same selected validation score as the fold score. Thus the fold is both a checkpoint-selection set and a reporting set. This produces potential optimistic selection bias relative to an outer test fold. The procedure is **not nested cross-validation**. Classical fixed fits have no analogous per-epoch checkpoint selection, so identical split membership alone does not make the two selection procedures identical.

Common neural settings are ten maximum epochs; batch size 16; evaluation batch size 32; accumulation 1; AdamW LR 2×10^{-5}; weight decay 0.01; 10% warmup; gradient norm cap 1; patience 2; zero loader workers; mixed precision disabled. E9 changes patience to 3, requires at least five epochs before stopping, and breaks equal micro-F1 ties with micro-AP. The training loop restores the best state after stopping. It does not choose the final epoch simply because it was last executed.

5.4.2 Stage 2: fixed approximately 80/10/10 evaluation

Models are initialized afresh and trained on the fixed training split. Transformers use validation micro-F1 to choose a checkpoint and then score the test split. ML models fit once using the fixed settings; validation is diagnostic rather than an epoch-selection mechanism. E4's implementation can subsequently tune label thresholds on final validation, but no E4 final run is present. All reported final Transformer experiments use threshold 0.5.

Stage 1 used the full population before Stage 2 splitting. Every final test sentence therefore participated in screening as training and/or validation evidence. Stage 2 test scores are **not independent of method selection**; they are exploratory results under a new fit and partition, not a clean confirmatory holdout. Figure 7 makes the dependence explicit.

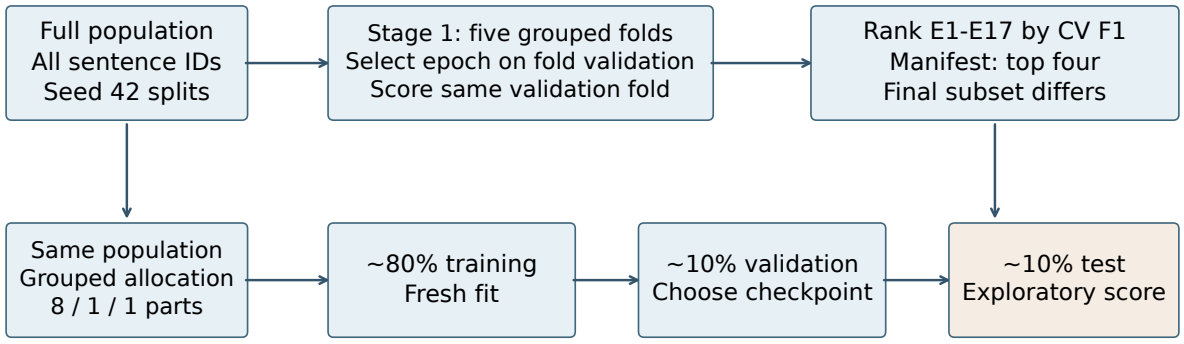


Figure 7: Actual two-stage protocol. Full-population screening and within-fold checkpoint selection limit independence. The later 10% test split is not an untouched outer test set.

The saved top-four selection manifest lists Python E15/E13/E17/E11, Java E15/E17/E9/E12, and Pharo E3/E9/E8/E12. The actual final neural set is instead E1/E3/E9/E15/E17 for every language. This is neither full E1–E17 coverage nor strict execution of each manifest's top four. The results are described as a fixed available subset; no undocumented selection rationale is invented. No corrected independent or nested experimental cohort was found.

5.5 Evaluation metrics

Let TP_ℓ, FP_ℓ, and FN_ℓ denote per-label counts, and let TP, FP, FN sum them across labels. Then

$$P_\ell = \frac{TP_\ell}{TP_\ell + FP_\ell}, \qquad R_\ell = \frac{TP_\ell}{TP_\ell + FN_\ell}, \tag{8}$$

$$F1_\ell = \frac{2TP_\ell}{2TP_\ell + FP_\ell + FN_\ell}, \qquad F1_{\text{micro}} = \frac{2TP}{2TP + FP + FN}, \tag{9}$$

$$F1_{\text{macro}} = \frac{1}{L}\sum_{\ell=1}^{L} F1_\ell, \qquad HL = \frac{1}{NL}\sum_{i,\ell} \mathbb{1}[y_{i\ell} \neq \hat{y}_{i\ell}]. \tag{10}$$

Zero-denominator precision, recall, and F1 are reported as zero by the implementations. Micro-F1 emphasizes frequent positive decisions; macro-F1 weights labels equally. Hamming loss counts both false positives and false negatives over all binary assignments and is lower when performance is better. It can look small even for an unhelpful all-negative predictor.

Average precision is a ranking metric. At successive distinct score thresholds k, $AP = \sum_k (R_k - R_{k-1})P_k$. Micro-AP flattens all sentence–label decisions into one ranking. Tied scores are grouped in the neural implementation; ML uses scikit-learn's AP. Per-label AP is undefined if no positives are observed, and only defined labels contribute to macro-AP. SVM decision scores support ranking evaluation without being probabilities; micro-AP can be sensitive to cross-label score scales.

The all-negative predictor has binary assignment accuracy 77.58% on Python, 83.19% on Java, and 83.46% on Pharo, despite zero positive recall and zero F1. These figures follow directly from dataset densities and

explain why accuracy alone is misleading. They are calculated diagnostics, not additional trained-model measurements. Subset accuracy, which requires the complete label vector to match, is a different metric and is not substituted for micro-F1.

6 Experimental Results

Our experimental cohort contains 51 Transformer screening runs (17 configurations $\times$ 3 languages), 18 classical/dummy screening runs (6 $\times$ 3), 15 Transformer final runs (5 $\times$ 3), and 15 classical final runs (5 $\times$ 3). All use base seed 42. Tables are generated from stored metrics; fold summaries and per-label counts were checked arithmetically. Aggregate F1 uses four decimal places, and reported percentage-point differences use those displayed values. Per-label tables use three decimals. Section 8 gives the common evaluation qualifications.

6.1 Traditional ML versus the Transformer reference: RQ1

Table 7 and Figure 8 compare the shared Stage 1 folds. The strongest traditional method by mean micro-F1 is Random Forest for Python (0.5882), logistic regression for Java (0.5972), and SGD for Pharo (0.6486). E1 reaches 0.6169, 0.5968, and 0.7044, respectively. Relative to each language's strongest traditional model, E1 differs by approximately $+2.87$, -0.04, and $+5.58$ percentage points. These are descriptive differences between fold means, not significant effects.

Table 7: Stage 1 micro-F1: mean and sample standard deviation across five folds. E1 uses fold validation for checkpoint selection.

Method	Python	Java	Pharo
Logistic regression	0.5833 ± 0.0126	0.5972 ± 0.0566	0.6438 ± 0.0223
Classifier chain	0.5623 ± 0.0193	0.5789 ± 0.0675	0.6209 ± 0.0247
Linear SVM	0.5730 ± 0.0095	0.5811 ± 0.0577	0.6372 ± 0.0210
SGD	0.5811 ± 0.0093	0.5932 ± 0.0556	0.6486 ± 0.0188
Random forest	0.5882 ± 0.0126	0.5726 ± 0.0405	0.6458 ± 0.0226
Majority dummy	0.0000 ± 0.0000	0.0000 ± 0.0000	0.0000 ± 0.0000
RoBERTa E1	0.6169 ± 0.0177	0.5968 ± 0.0688	0.7044 ± 0.0374

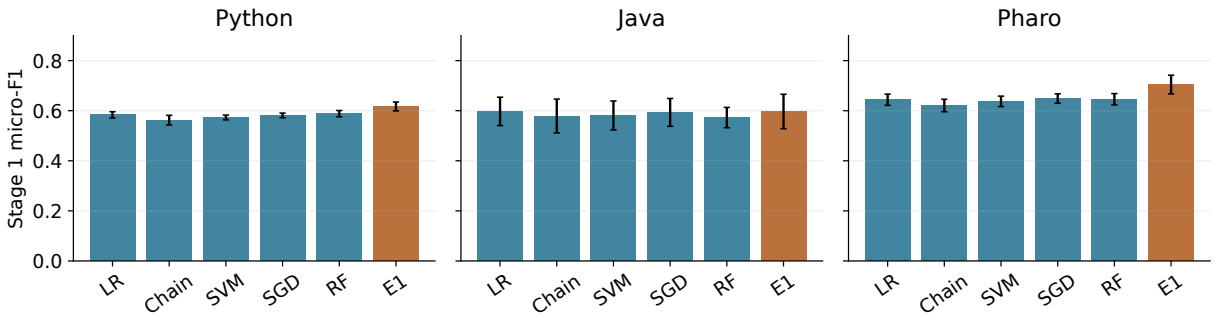


Figure 8: Five traditional methods versus RoBERTa E1 on Stage 1 micro-F1. Error bars are sample SD across five folds, not confidence intervals. LR denotes logistic regression.

The Java logistic baseline also has macro-F1 0.6390 compared with 0.6273 for E1. The difference suggests that the more expensive reference encoder does not dominate lexical models under every metric. Pharo shows a clearer aggregate separation: E1 macro-F1 is 0.6053 versus 0.5604 for SGD, though this still does not prove better performance on every label. Full multi-metric screening tables are included in Appendix A.

Answer to RQ1. Traditional TF-IDF models are competitive with E1 on Java and provide meaningful baselines elsewhere. E1 has higher observed screening micro-F1 than all five traditional methods on Python and Pharo.

6.2 Transformer configuration comparison: RQ2

Table 8 covers every E1–E17 screening configuration. E15 (CodeBERT R-Drop) leads Python at 0.6479 and Java at 0.6225. E3 (RoBERTa focal) leads Pharo at 0.7152. Their improvements over E1 are about 3.10, 2.57, and 1.08 percentage points. There is no universal language-independent winner.

Table 8: Transformer screening mean and sample SD. E4 shares E1 scores in all three languages and is omitted because threshold tuning is deferred. All 51 run records remain available.

Configuration	Python	Java	Pharo
E1	0.6169 ± 0.0177	0.5968 ± 0.0688	0.7044 ± 0.0374
E2	0.6161 ± 0.0199	0.5981 ± 0.0593	0.6912 ± 0.0445
E3	0.6219 ± 0.0256	0.5954 ± 0.0590	0.7152 ± 0.0233
E5	0.6080 ± 0.0213	0.6055 ± 0.0488	0.6992 ± 0.0318
E6	0.6158 ± 0.0167	0.6115 ± 0.0688	0.7036 ± 0.0278
E7	0.6081 ± 0.0102	0.5918 ± 0.0564	0.6736 ± 0.0208
E8	0.6271 ± 0.0285	0.6065 ± 0.0521	0.7104 ± 0.0307
E9	0.6211 ± 0.0268	0.6125 ± 0.0555	0.7124 ± 0.0304
E10	0.6260 ± 0.0196	0.5957 ± 0.0462	0.6950 ± 0.0253
E11	0.6397 ± 0.0181	0.5980 ± 0.0300	0.6805 ± 0.0358
E12	0.6132 ± 0.0181	0.6117 ± 0.0519	0.7068 ± 0.0367
E13	0.6455 ± 0.0163	0.6001 ± 0.0284	0.6893 ± 0.0347
E14	0.6307 ± 0.0196	0.6032 ± 0.0205	0.6767 ± 0.0282
E15	0.6479 ± 0.0237	0.6225 ± 0.0416	0.6956 ± 0.0450
E16	0.6376 ± 0.0175	0.5979 ± 0.0404	0.6748 ± 0.0202
E17	0.6452 ± 0.0219	0.6154 ± 0.0259	0.6993 ± 0.0287

E11, the CodeBERT BCE backbone comparator, scores 0.6397 on Python, 0.5980 on Java, and 0.6805 on Pharo. It is higher than E1 on Python, nearly equal on Java, and lower on Pharo. Comparing E15 with E11 gives R-Drop-associated increments of about 0.82, 2.45, and 1.51 points within the CodeBERT backbone in this cohort. Those matched comparisons are more informative than attributing the entire E15–E1 difference to regularization.

Loss changes do not improve every language. E3 is the leading Pharo screening configuration but slightly below E1 on Java. Weighted BCE and ASL alter precision/recall tradeoffs rather than uniformly raising micro-F1. E4 is identical to E1 in all reported screening metrics because screening defers threshold tuning; the duplicate row is omitted from the tables rather than treated as a separate gain. More complex graph, attention, and layer-wise optimization variants also fail to dominate all simpler alternatives.

6.3 Final evaluation

Tables 9–11 contain all available final results, including the five traditional models. They report micro-F1, macro-F1, Hamming loss, and micro-AP. They do not fill in the twelve missing neural configurations per language.

Among final neural runs, E15 is highest on Python (micro-F1 0.6541, macro-F1 0.5965), E17 on Java (0.7034, 0.7342), and E1 on Pharo (0.7371, 0.6720). Across all available methods, these are also the highest test micro-F1 values. However, Java's classifier chain has macro-F1 0.7470 and micro-F1 0.6964, illustrating a different tradeoff from the neural test leader.

Table 9: Python Stage 2 test results. HL is Hamming loss (lower is better).

Method	Micro-F1	Macro-F1	HL	Micro-AP
E15	0.6541	0.5965	0.1437	0.7585
E9	0.6285	0.5634	0.1570	0.6779
E17	0.6248	0.5669	0.1492	0.7474
E1	0.6074	0.5547	0.1656	0.6571
E3	0.5900	0.5496	0.1727	0.6880
Linear SVM	0.5704	0.5295	0.1906	0.6326
SGD	0.5690	0.5268	0.1953	0.6395
Logistic regression	0.5675	0.5244	0.1953	0.6392
Classifier chain	0.5577	0.5275	0.1945	0.5772
Random forest	0.5421	0.5019	0.1953	0.6408

Table 10: Java Stage 2 test results. HL is Hamming loss (lower is better).

Method	Micro-F1	Macro-F1	HL	Micro-AP
E17	0.7034	0.7342	0.0921	0.8040
Classifier chain	0.6964	0.7470	0.1004	0.7503
SGD	0.6950	0.7441	0.1015	0.7751
E1	0.6947	0.6946	0.0945	0.7580
E15	0.6914	0.7284	0.0933	0.7986
Logistic regression	0.6871	0.7344	0.1027	0.7773
E3	0.6839	0.7111	0.1009	0.7910
Linear SVM	0.6838	0.7335	0.1015	0.7412
E9	0.6762	0.7371	0.1068	0.7465
Random forest	0.6360	0.6495	0.1216	0.7430

Table 11: Pharo Stage 2 test results. HL is Hamming loss (lower is better).

Method	Micro-F1	Macro-F1	HL	Micro-AP
E1	0.7371	0.6720	0.0828	0.7990
E9	0.7157	0.6058	0.0909	0.7766
E17	0.7080	0.6283	0.0917	0.8084
E15	0.7050	0.6194	0.0917	0.8013
E3	0.6919	0.6015	0.0990	0.7956
Random forest	0.6447	0.5607	0.1136	0.7126
SGD	0.6420	0.5788	0.1177	0.6618
Logistic regression	0.6400	0.5653	0.1169	0.6691
Linear SVM	0.6308	0.5601	0.1169	0.6545
Classifier chain	0.6161	0.5197	0.1274	0.6224

6.4 Cross-validation versus test ranking

Figure 9 compares the five neural configurations having both stages. Python E15 remains first in this subset. Java E17 overtakes E15; Pharo's E1 overtakes its screening leader E3. CV and final test measure different fitted models and evaluation sets: CV averages selected fold-validation scores, whereas final training uses a new training/validation/test partition. Both training composition and checkpoint selection can affect ranking, as can sampling variation. The observed changes do not establish intrinsic model instability.

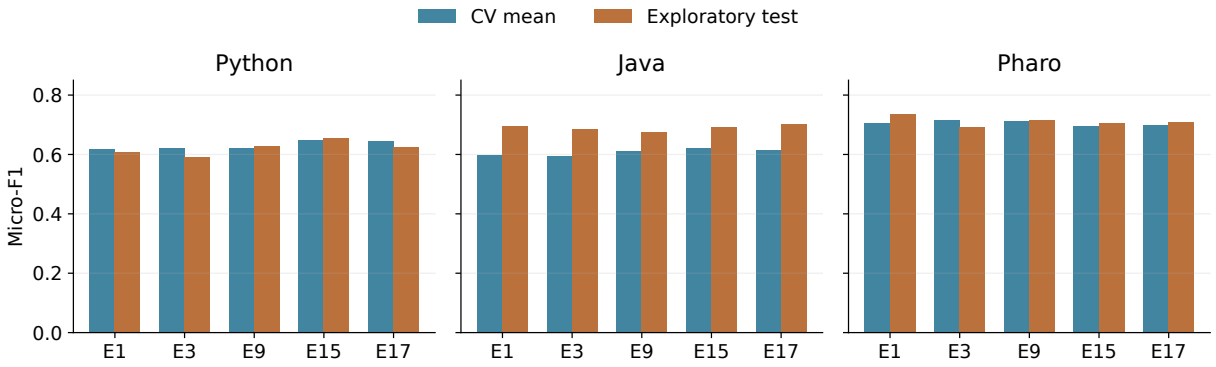


Figure 9: Stage 1 means and Stage 2 test micro-F1 for the five evaluated Transformers. Scores are not independent replications and differences must not be read as generalization-gap estimates.

Answer to RQ2. E15 leads full neural screening for Python and Java, while E3 leads Pharo. Within the available final subset, E15/E17/E1 yield the highest observed test micro-F1 for Python/Java/Pharo. The ranking depends on the language and evaluated subset.

6.5 Per-label performance: RQ3

Tables 12 and 13 compare E1 with the highest observed final micro-F1 Transformer in the corresponding language. This retrospective comparator is descriptive. Figure 10 and the other heatmaps show all five final configurations so that regressions are not hidden by selecting a single winner.

Table 12: Python test label performance. The highest observed test micro-F1 Transformer is E15; comparator choice is retrospective and descriptive.

Label	Support	E1 P	E1 R	E1 F1	E15 P	E15 R	E15 F1
DevelopmentNotes	31	0.429	0.290	0.346	0.389	0.226	0.286
Expand	51	0.556	0.294	0.385	0.579	0.431	0.494
Parameters	80	0.642	0.650	0.646	0.743	0.688	0.714
Summary	45	0.771	0.600	0.675	0.680	0.756	0.716
Usage	80	0.685	0.762	0.722	0.862	0.700	0.772

On Python, E15 raises aggregate micro-F1 from 0.6074 to 0.6541 and macro-F1 from 0.5547 to 0.5965. Yet DevelopmentNotes F1 falls from 0.346 to 0.286 (31 positives), while Expand rises from 0.385 to 0.494 and Parameters from 0.646 to 0.714. Thus even a simultaneous micro/macro improvement is compatible with a minority-label regression.

On Java, E17 improves micro-F1 over E1 from 0.6947 to 0.7034 and macro-F1 from 0.6946 to 0.7342. The rational label rises from 0.182 to 0.571, while Expand falls from 0.630 to 0.474 and Pointer from 0.724 to 0.678. Ownership is 1.000 for both, based on eleven positives. Per-label recall and precision in Table 13 show whether a score change reflects missed positives or additional predicted positives; no sentence-level explanation is inferred from counts alone.

Table 13: Java test label performance. The highest observed test micro-F1 Transformer is E17; comparator choice is retrospective and descriptive.

Label	Support	E1 P	E1 R	E1 F1	E17 P	E17 R	E17 F1
Expand	63	0.756	0.540	0.630	0.676	0.365	0.474
Ownership	11	1.000	1.000	1.000	1.000	1.000	1.000
Pointer	35	0.913	0.600	0.724	0.833	0.571	0.678
deprecation	12	1.000	0.833	0.909	0.917	0.917	0.917
rational	28	0.600	0.107	0.182	0.857	0.429	0.571
summary	42	0.544	0.738	0.626	0.653	0.762	0.703
usage	91	0.791	0.791	0.791	0.760	0.835	0.796

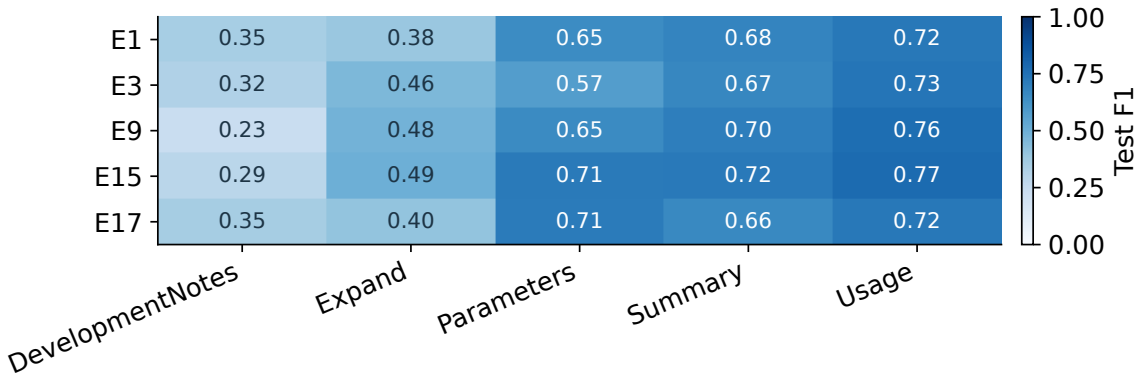


Figure 10: Python test F1 by label and final Transformer configuration. Every value is read from stored per-label metrics; supports are given in Table 12.

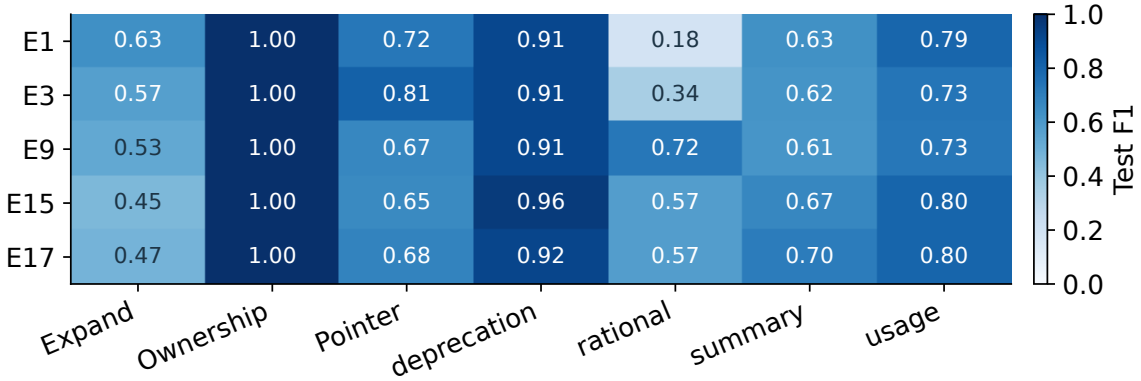


Figure 11: Java test F1 by label and configuration. High Ownership scores are based on only eleven positive test assignments.

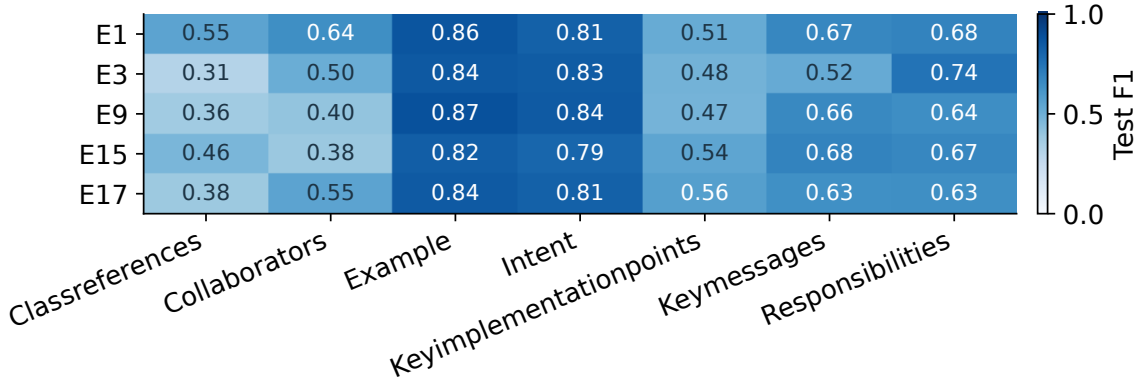


Figure 12: Pharo test F1 by label and configuration. E1 leads aggregate test micro-F1, while some of its individual label scores are surpassed.

On Pharo, E1 has higher aggregate scores than E15, but E15 has higher Keyimplementationpoints F1 (0.537 versus 0.512) and Keymessages F1 (0.677 versus 0.667). Classreferences support is only seven; its F1 ranges from 0.308 for E3 to 0.545 for E1 across the five final configurations. Such rare-category variation is practically relevant but imprecisely estimated. Full E1 per-label precision/recall and support for Pharo are supplied in the appendix.

Answer to RQ3. No. Aggregate improvement does not imply improvement for each label. The stored metrics contain explicit counterexamples in Python and Java, and Pharo shows label-level gains within an aggregate-lower configuration. Neither micro- nor macro-F1 should replace per-label assessment when rare information types matter.

6.6 Learning dynamics and early stopping

Figure 13 uses the saved logistic-regression curve files. Each point is a separate refit at a specified maximum L-BFGS iteration count, not a logged epoch or a guaranteed prefix of one long optimization trajectory. On Python, training Hamming loss falls from 0.3009 at the first limit to 0.0178 at the final limit; validation micro-F1 settles near 0.5654. The corresponding final validation values are 0.6046 for Java and 0.6294 for Pharo. Strong training fit with a lower validation plateau indicates limited generalization benefit from further optimization; the different metrics must not be subtracted as if they were the same loss.

Figure 14 shows the actual E1/E15 training histories. For Java E15, validation micro-F1 peaks at epoch 3 (0.6402) and declines through epochs 4–5 while training objective continues falling; patience 2 stops the run after epoch 5 and restores epoch 3. Java E1 similarly selects epoch 6 and finishes at epoch 8. Python E1/E15 both select epoch 9 out of 10; Pharo E1 selects epoch 8 out of 10 and E15 epoch 6 out of 8. These trajectories demonstrate why final-epoch weights are not necessarily the selected weights. A falling training objective with weaker validation scores is consistent with overfitting, though one split does not establish a universal instability property. E1 and E15 have different objectives, so their training-loss magnitudes are not directly comparable.

6.7 Illustrative comments and quantitative difficult-label analysis

Table 14 gives actual dataset sentences and their annotated labels. Python sentence 19 has both Expand and Usage; Java sentence 61 combines rational and summary. Such examples make the multi-label formulation concrete and show that annotation can depend on fragments and surrounding context. They are not claimed to be model errors or model-generated explanations.

The Java rational label provides a quantitative difficult case. E1 detects 3 of 28 positives (recall 0.1071), predicts 5 positives, and therefore has 25 false negatives and 2 false positives. E17 detects 12 of 28 (recall

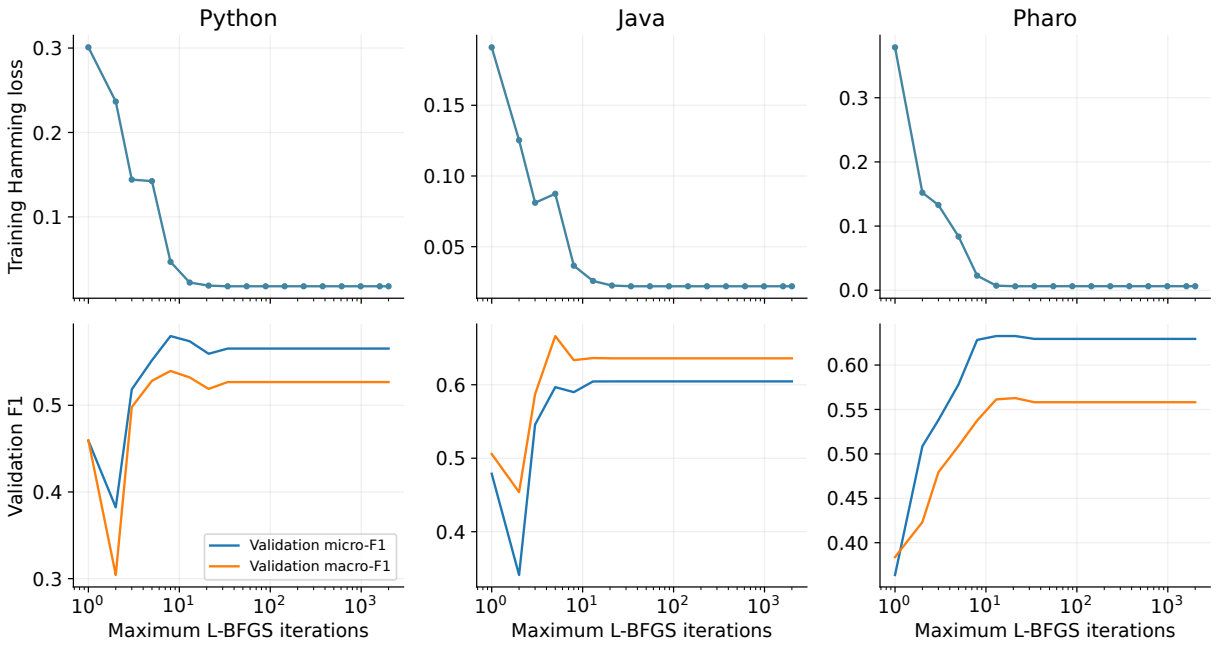


Figure 13: Stored logistic-regression refit diagnostics. Top: training Hamming loss; bottom: validation F1. The horizontal axis is the maximum solver iteration count on a log scale, not a neural epoch.

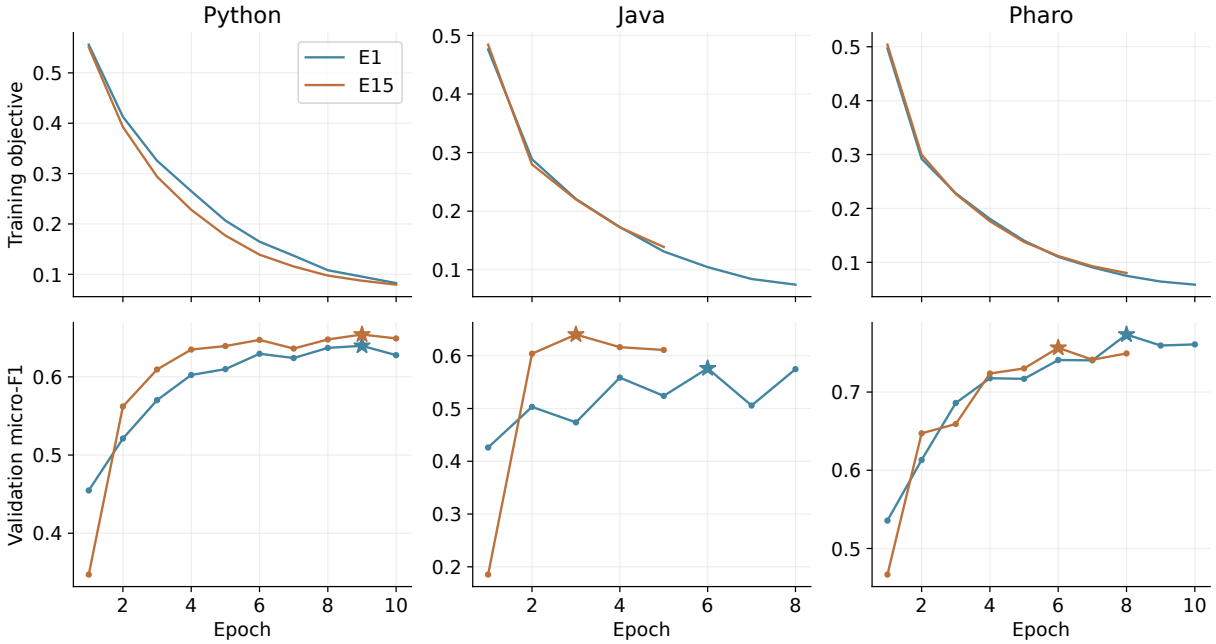


Figure 14: Saved neural histories for E1 and E15. Stars mark selected validation epochs. Loss values are objective-specific; the bottom row compares the common validation micro-F1 metric.

Table 14: Actual annotated sentences illustrating information types. These are dataset examples, not model errors or predictions.

Language	ID	Sentence	Annotated labels
Python	8	only provides a label and a dict of models.	Parameters
Python	19	decorated function in the instance itself, used as a context manager.	Expand, Usage
Java	61	contains all the state data that needs to be stored persistently for an application	rational, summary
Pharo	3	i contain examples of different types of blarrowheadelement	Classreferences, Responsibilities

0.4286), predicts 14 positives, and has 16 false negatives and 2 false positives. Its F1 rises from 0.1818 to 0.5714 through more true positives without an increase in false-positive count on this split. This is a count-level explanation supported by stored confusion evidence; the missing row-level predictions prevent identifying which sentences changed.

6.8 Published benchmark context

The organizers' values in Table 15 average 19 binary category F1 scores on official partitions [6]. Their ranking weights average F1 by 0.75 and the proportion of improved categories by 0.25. Our per-language micro-F1 uses different splits and aggregation, so these published values locate the task without supplying a directly comparable superiority margin.

Table 15: Published NLBSE'23 results from the organizers' Table III; context only, not directly comparable with our grouped splits.

Method	Average precision	Average recall	Average F1
Official RF baseline	0.439	0.245	0.309
Submitted logistic regression	0.540	0.560	0.547
CodeT5	0.728	0.606	0.657
STACC	0.710	0.794	0.744

7 Discussion

7.1 When traditional ML is competitive

The Java results demonstrate that an encoder is not automatically preferable for these partitions. Word/character TF-IDF with logistic regression matches E1 screening micro-F1 to within a few hundredths of a percentage point and has higher macro-F1. Classical final results also remain close to the neural leaders, with the chain producing the highest observed Java macro-F1 among available final methods. This supports retaining lexical baselines in model selection. It does not establish deployment efficiency: the repository lacks a controlled comparison of inference latency, memory, and hardware-normalized energy or cost.

The larger reference-model advantage on Pharo suggests that the balance between lexical cues and contextual representation differs across the corpora. Different taxonomies, text distributions, and support profiles vary together with programming language. It is therefore not possible to isolate programming language as the cause. The present evidence supports a corpus-specific comparison, not a general ranking of how well Transformers understand the three languages.

7.2 Backbones, objectives, and interactions

The experiment matrix contains useful contrasts, including E1/E11 for backbone changes and E11/E15 for CodeBERT R-Drop. However, E15 versus E1 changes both backbone and consistency regularization, and E16 combines learning-rate changes with R-Drop. E9 also has a different stopping policy. A headline comparison between each language's strongest configuration and E1 cannot allocate gains uniquely among backbone, objective, stopping, and optimization effects. No balanced factorial study or controlled causal analysis is present.

The observation that CodeBERT leads screening on Python but its BCE variant trails RoBERTa on Pharo is compatible with several hypotheses: differences in lexical cues, pretraining relevance, head fitting, and label-support interactions. None is verified by the available artifacts. The report therefore separates observed score differences from proposed explanations. Attention weights, graph design, or model complexity alone do not establish interpretable reasoning about comments.

7.3 Aggregate and rare-label performance

Sparse positive assignments create different objectives for different use cases. A documentation-search tool prioritizing common usage information may reasonably emphasize micro-F1. A maintenance tool requiring reliable detection of DevelopmentNotes, deprecation, or Classreferences needs label-specific precision and recall. Macro-F1 gives each category equal weight but can still mask a particular regression by averaging improvements elsewhere. The explicit Python DevelopmentNotes regression under E15 is a reason to inspect the relevant label, even when both aggregate F1 measures increase.

Changing a threshold can alter this tradeoff without changing the encoder. The fixed 0.5 threshold is an operational choice, not a demonstrated optimum for every label. E4 implements validation-based tuning but was not evaluated in the final cohort. Its unchanged screening scores should not be used to dismiss threshold calibration. A future application would need validation criteria reflecting the actual costs of false positives and false negatives.

7.4 Practical implications and generalization

A defensible workflow would keep a simple lexical baseline, compare neural candidates using training-only selection, inspect label-level outcomes, and evaluate a frozen configuration on genuinely unseen groups. The current repository already supplies shared data validation and transparent result contracts that would support that extension. The new dataset adapters extend its storage design; a deployed inference service and independent project-level evaluation remain future work.

The appropriate practical claim is that these methods can learn information-type associations on this dataset under the recorded protocol. Reliability on new projects, different comment conventions, other languages, and developer workflows is unresolved. Potential downstream gains in maintenance productivity or documentation quality require their own evaluation.

8 Limitations and Threats to Validity

8.1 Selection and statistical validity

Stage 1 screens methods on the full population, including sentences later assigned to Stage 2 test. Final testing is therefore exploratory rather than independent confirmation. Neural CV selects an epoch on the same validation fold used for its reported score. This can introduce optimistic checkpoint-selection bias and differs from the fixed classical fitting process. A future outer holdout should exclude test data

from configuration and epoch selection; inner validation or a fixed training schedule would address the checkpoint issue.

The cohort has one base seed, 42; fold seeds 42–46 are not repeated full-protocol seeds. Screening seventeen configurations creates multiple-comparison opportunities, and fold SD is descriptive variation rather than a confidence interval. No statistical significance is claimed. Small label supports make some F1 values sensitive to a few decisions. Repeated protocol seeds and paired uncertainty analysis are needed to establish whether small differences persist.

8.2 Data and external validity

Exact whitespace-normalized duplicates stay in one group; fold-local vectorizers, weights and graphs avoid fitting these statistics on validation/test labels. However, near duplicates and class/project conventions remain possible. Train/test class overlap is substantial, and project identity is not a grouping variable. Generalization to new projects or developer workflows remains untested. Different language taxonomies/-supports also prevent attributing corpus-level score differences to language alone. Information-type classification does not measure comment correctness or usefulness.

8.3 Metrics and experimental coverage

The final neural subset is E1/E3/E9/E15/E17, not all seventeen configurations or each language's saved top four. Missing final results are not inferred. E4 screening uses E1's threshold 0.5; its final threshold-tuning experiment is absent. Threshold sensitivity and score scales can affect F1 and pooled AP, motivating our precision/recall, macro-F1 and support analysis. Published scores use official splits and aggregation, as distinguished in Section 6.8.

8.4 Artifact and software scope

Aggregate/per-label metrics are available, but original sentence predictions and distributed checkpoints are not. These support numerical error counts but cannot identify actual sentence-level false positives/negatives or replay decisions. Section 6.7 labels its annotated examples as illustrations, not model errors. Exact historical package/Python versions and GPU/driver/cost measurements are missing, so runtime is not a controlled efficiency comparison.

The new adapters cover staged-dataset persistence; they do not convert every tokenizer, checkpoint or plot writer into a switchable backend. They are verified against the same data without rerunning archived experiments. Future releases should preserve predictions, model files, environment metadata and project-grouped split IDs before evaluating frozen candidates on an independent outer protocol.

9 Reproducibility

9.1 Source, data and environment

The project separates `configs/`, `data/`, `src/`, `experiments/`, `scripts/`, `notebooks/` and `results/`. Appendix C maps report sections to artifacts. The original manuscript and scientific metrics are preserved; new storage adapters and report assets are revision-time additions rather than new model experiments.

The prepared data roots are `data/k_fold_data/` and `data/full_training_data/`. Raw category-wise CSVs are tracked directly under `data/`; the default downloader directory is not required for using these copies. Configuration files pin the upstream dataset commit, while saved manifests identify populations and splits.

The requirements specify torch $\geq 2.4, < 3$, transformers $\geq 4.40, < 5$, PyYAML $\geq 6, < 7$, tensorboard $\geq 2.14, < 3$, matplotlib $\geq 3.7, < 4$, tqdm $\geq 4.66, < 5$, NumPy $\geq 1.24, < 3$, pandas $\geq 2.0, < 3$, scikit-learn $\geq 1.4, < 2$, and joblib $\geq 1.3, < 2$. The current verification environment uses Python 3.12.10 on Windows. Record exact dependency/device versions before future training; pretrained weights require network access or a local cache.

9.2 Installation and dataset preparation

From the repository root:

```
python -m pip install -r requirements.txt
python -m src.data.prepare_two_stage --raw-dir data --output-dir report_rebuilt/reproduction_data
     --folds 5 --seed 42
```

The second command writes a new tree and preserves archived splits. Pass its `k_fold_data` or `full_training_data` subdirectory via `--data-dir` to use regenerated data; otherwise runners use the original prepared roots. The verified regeneration recreates all six staged dataset byte fingerprints, sentence orders, labels and fold definitions.

9.3 Training and evaluation

These commands invoke real entry points; they are reproduction instructions, not additional training claimed in this revision. Choose fresh output paths for each run.

```
python -m experiments.ml.train_baseline --stage kfold --language all --seed 42 --output-dir
    results/reproduction_ml_cv
python -m experiments.ml.train_baseline --stage final --language all --methods ovr_logreg
    classifier_chain_logreg ovr_linear_svc ovr_sgd ovr_random_forest --seed 42 --output-dir
    results/reproduction_ml_final
python -m experiments.dl.train_exp1 --stage kfold --language python --device cpu --seed 42 --
    output-dir results/reproduction_e1_cv/python
python -m experiments.dl.train_exp1 --stage final --language python --device cpu --seed 42 --
    output-dir results/reproduction_e1_final/python
```

Use an available CUDA device for practical neural execution. Both CLI families accept `--storage memory` to load a snapshot through MemoryStore; `--storage file` is the default. The same parsing, validation and model code is used. Result export directories remain on the filesystem.

In a Bash-capable environment, complete screening and the historical final neural subset can be run with:

```
bash scripts/train_kfold.sh --seed 42 --device cuda:0 --output-dir results/reproduction_dl_cv
bash scripts/train_full.sh --experiments 1,3,9,15,17 --seed 42 --device cuda:0 --output-dir
    results/reproduction_dl_final
```

Alternatively, the `src.evaluation.two_stage_results` module's `select` command writes a top-four manifest for `train_full.sh --selection`; that follows a different candidate set from the historical fixed five. All neural experiments use base seed 42, with fold seeds base plus fold index. The evaluation design is specified in Section 5 and qualified in Section 8.

9.4 Artifacts, visualizations and software checks

Runs export configurations, label order, data manifests, metrics and plots. CV outputs include fold records; final neural code can save checkpoints/thresholds, and ML code can export prediction CSVs and serialized models. The distributed scientific evidence is the stored JSON/CSV metadata and figures; availability limits are listed in Section 8.

```
python report_rebuilt/scripts/audit.py
python report_rebuilt/scripts/research_assets.py
python report_rebuilt/scripts/diagrams.py
python -m unittest discover -s tests -v
```

The report scripts generate tables, EDA, annotated examples and learning diagnostics from existing data/history files without fitting research models. Storage tests cover both adapter contracts and all six actual staged datasets. Offline training tests use tiny mocked backbones; they are software checks rather than research replications. Test/build outcomes are recorded in `validation.md`.

Three EDA notebooks include saved outputs. The notebook exporter is `scripts/export_eda_figures.py`; its raw-data and local tokenizer prerequisites are documented in `notebooks/README.md`. The report's direct EDA script uses the same prepared sentence populations and is independent of executing those notebooks.

9.5 Report build

The PDF is committed as `report_rebuilt/main.pdf`. To rebuild, run XeLaTeX, BibTeX, then XeLaTeX twice from `report_rebuilt/`, or upload the source bundle to Overleaf and select XeLaTeX. The cover follows the original manuscript's institution, instructor and team details. Times New Roman is selected when installed, with TeX Gyre Termes as fallback. Numerical checks and visual review accompany the build; source files remain editable.

10 Conclusion

We compared five TF-IDF learning methods and seventeen RoBERTa/CodeBERT configurations on 6,738 multi-label code comment sentences. The results show that model selection should consider the language-specific corpus and the information types needed by the application.

RQ1: Traditional models are competitive on Java: logistic regression essentially matches RoBERTa E1 screening micro-F1 and exceeds its macro-F1. E1 has higher observed micro-F1 than all five traditional methods on Python and Pharo.

RQ2: CodeBERT R-Drop leads screening on Python and Java; RoBERTa focal leads Pharo. In the available final subset, the highest observed test micro-F1 belongs to E15, E17 and E1 for Python, Java and Pharo, respectively. Combined comparisons do not isolate one intervention as the cause of improvement.

RQ3: Aggregate gains do not guarantee improvement for every label. Python DevelopmentNotes regresses under E15, while Java E17 loses F1 on Expand and Pointer despite higher overall scores. Per-label support, precision and recall are necessary companions to aggregate F1.

Our software contribution includes shared dataset validation, interchangeable dataset stores, configuration-driven models and reproducible visualizations. This is an empirical comparison rather than a new architecture. Following Section 8, future work should prioritize independent project-grouped evaluation, repeated seeds, sentence-level predictions and validation-only threshold calibration to establish how these tradeoffs transfer to practical comment-management tasks.

Acknowledgements

We thank Dr. Phuong Nguyen for supervising the Seminar 2 project and the NLBSE'23 organizers and dataset contributors for making the benchmark and baseline materials publicly available.

References

[1] Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. STACC: Code comment classification using SentenceTransformers. arXiv:2302.13149, 2023. https://arxiv.org/abs/2302.13149.

[2] Emanuel Ben-Baruch, Tal Ridnik, Nadav Zamir, Asaf Noy, Itamar Friedman, Matan Protter, and Lihi Zelnik-Manor. Asymmetric loss for multi-label classification. arXiv:2009.14119, 2020. arXiv version record. https://arxiv.org/abs/2009.14119.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019. https://aclanthology.org/N19-1423/.

[4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. CodeBERT: A pre-trained model for programming and natural languages. arXiv:2002.08155, 2020. https://arxiv.org/abs/2002.08155.

[5] Dorsaf Haouari, Houari Sahraoui, and Philippe Langlais. How good is your comment? a study of comments in java programs. In *International Symposium on Empirical Software Engineering and Measurement*, 2011. Author manuscript: https://www.iro.umontreal.ca/~felipe/bib2webV0.81/cv/papers/paper-esem-2011.pdf.

[6] Rafael Kallis, Maliheh Izadi, Luca Pascarella, Oscar Chaparro, and Pooja Rani. The NLBSE'23 tool competition. In *IEEE/ACM International Workshop on Natural Language-Based Software Engineering*, pages 1–8, 2023. Author manuscript: https://ojcchar.github.io/files/24-nlbse23-tools.pdf.

[7] Xiaobo Liang, Lijun Wu, Juntao Li, Yue Wang, Qi Meng, Tao Qin, Wei Chen, Min Zhang, and Tie-Yan Liu. R-Drop: Regularized dropout for neural networks. arXiv:2106.14448, 2021. https://arxiv.org/abs/2106.14448.

[8] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. arXiv:1708.02002, 2017. https://arxiv.org/abs/1708.02002.

[9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692, 2019. https://arxiv.org/abs/1907.11692.

[10] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1711.05101.

[11] NLBSE'23 Tool Competition Organizers. NLBSE'23 tool competition: Code comment classification. Official dataset and baseline repository, 2023. Accessed 5 November 2026. https://github.com/nlbse2023/code-comment-classification.

[12] Luca Pascarella and Alberto Bacchelli. Classifying code comments in java open-source software systems. In *IEEE/ACM International Conference on Mining Software Repositories*, 2017. https://sback.it/publications/msr2017a.pdf.

[13] Pooja Rani, Suada Abukar, Nataliia Stulova, Alexandre Bergel, and Oscar Nierstrasz. Do comments follow commenting conventions? a case study in java and python. arXiv:2108.10766, 2021. https://arxiv.org/abs/2108.10766.

[14] Pooja Rani, Arianna Blasi, Nataliia Stulova, Sebastiano Panichella, Alessandra Gorla, and Oscar Nierstrasz. A decade of code comment quality assessment: A systematic literature review. *Journal of Systems and Software*, 195:111515, 2023. https://arxiv.org/abs/2209.08165.

[15] Pooja Rani, Sebastiano Panichella, Manuel Leuenberger, Mohammad Ghafari, and Oscar Nierstrasz. What do class comments tell us? an investigation of comment evolution and practices in pharo smalltalk. arXiv:2005.11583, revised version, 2021. https://arxiv.org/abs/2005.11583.

[16] Jesse Read, Bernhard Pfahringer, Geoff Holmes, and Eibe Frank. Classifier chains for multi-label classification. In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD)*, pages 254–269, 2009. Author manuscript record: https://hdl.handle.net/10289/3259.

[17] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015.

[18] Sida Wang and Christopher D. Manning. Baselines and bigrams: Simple, good sentiment and topic classification. In *Proceedings of ACL, Volume 2: Short Papers*, pages 90–94, 2012. https://aclanthology.org/P12-2018/.

[19] Min-Ling Zhang and Zhi-Hua Zhou. A review on multi-label learning algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 26(8):1819–1837, 2014. Author manuscript: https://cs.nju.edu.cn/zhouzh/zhouzh.files/publication/tkde13rev.pdf.

A Complete Screening Metrics

These tables give all distinct method scores using mean fold metrics. E4 shares E1's screening values and is omitted; all 51 original neural run records are preserved. No cross-language mean is reported because label spaces and distributions differ.

Table 16: Python Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted.

Method	Micro-F1	Macro-F1	HL	Micro-AP
E15	0.6479	0.5861	0.1456	0.7378
E13	0.6455	0.5893	0.1499	0.7297
E17	0.6452	0.5938	0.1489	0.7360
E11	0.6397	0.5740	0.1494	0.7253
E16	0.6376	0.5581	0.1453	0.7347
E14	0.6307	0.5480	0.1504	0.7215
E8	0.6271	0.5683	0.1555	0.7098
E10	0.6260	0.5363	0.1504	0.7178
E3	0.6219	0.5653	0.1595	0.7101
E9	0.6211	0.5723	0.1633	0.7009
E1	0.6169	0.5612	0.1645	0.6982
E2	0.6161	0.5834	0.1822	0.6821
E6	0.6158	0.5563	0.1619	0.6920
E12	0.6132	0.5597	0.1638	0.7014
E7	0.6081	0.5802	0.2182	0.7132
E5	0.6080	0.5451	0.1652	0.7015
Random forest	0.5882	0.5428	0.1760	0.6685
Logistic regression	0.5833	0.5470	0.1885	0.6511
SGD	0.5811	0.5459	0.1925	0.6447
Linear SVM	0.5730	0.5381	0.1915	0.6329
Classifier chain	0.5623	0.5233	0.1951	0.5955
Majority dummy	0.0000	0.0000	0.2242	0.2242

Table 17: Java Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted.

Method	Micro-F1	Macro-F1	HL	Micro-AP
E15	0.6225	0.6495	0.1170	0.6820
E17	0.6154	0.6506	0.1206	0.6893
E9	0.6125	0.6469	0.1243	0.6359
E12	0.6117	0.6497	0.1273	0.6337
E6	0.6115	0.6557	0.1233	0.6485
E8	0.6065	0.6361	0.1234	0.6647
E5	0.6055	0.6406	0.1232	0.6641
E14	0.6032	0.6439	0.1219	0.6799
E13	0.6001	0.6422	0.1231	0.6645
E2	0.5981	0.6493	0.1583	0.6449
E11	0.5980	0.6262	0.1222	0.6705
E16	0.5979	0.6286	0.1190	0.6910
Logistic regression	0.5972	0.6390	0.1315	0.6737
E1	0.5968	0.6273	0.1297	0.6424
E10	0.5957	0.6324	0.1264	0.6697
E3	0.5954	0.6312	0.1277	0.6507
SGD	0.5932	0.6363	0.1354	0.6699
E7	0.5918	0.6337	0.1739	0.6524
Linear SVM	0.5811	0.6241	0.1354	0.6417
Classifier chain	0.5789	0.6282	0.1387	0.6354
Random forest	0.5726	0.6146	0.1417	0.6205
Majority dummy	0.0000	0.0000	0.1681	0.1681

Table 18: Pharo Stage 1 fold means. HL is Hamming loss (lower is better). E4 shares E1 screening scores and is omitted.

Method	Micro-F1	Macro-F1	HL	Micro-AP
E3	0.7152	0.6267	0.0906	0.7961
E9	0.7124	0.6094	0.0909	0.7868
E8	0.7104	0.6071	0.0900	0.7833
E12	0.7068	0.6245	0.0938	0.7753
E1	0.7044	0.6053	0.0938	0.7699
E6	0.7036	0.6276	0.0949	0.7755
E17	0.6993	0.6188	0.0943	0.7740
E5	0.6992	0.6271	0.0965	0.7700
E15	0.6956	0.5857	0.0938	0.7738
E10	0.6950	0.5761	0.0928	0.7873
E2	0.6912	0.6358	0.1137	0.7611
E13	0.6893	0.5860	0.0971	0.7594
E11	0.6805	0.5706	0.0973	0.7545
E14	0.6767	0.5675	0.0981	0.7538
E16	0.6748	0.5374	0.0975	0.7702
E7	0.6736	0.6160	0.1352	0.8067
SGD	0.6486	0.5604	0.1149	0.7043
Random forest	0.6458	0.5196	0.1105	0.7093
Logistic regression	0.6438	0.5545	0.1164	0.7122
Linear SVM	0.6372	0.5474	0.1149	0.7063
Classifier chain	0.6209	0.5279	0.1262	0.6487
Majority dummy	0.0000	0.0000	0.1654	0.1654

B Pharo Reference Per-label Results

Table 19: Pharo test label performance. The highest observed test micro-F1 Transformer is E1; comparator choice is retrospective and descriptive.

Label	Support	E1 P	E1 R	E1 F1
Classreferences	7	0.750	0.429	0.545
Collaborators	13	0.778	0.538	0.636
Example	75	0.835	0.880	0.857
Intent	21	0.810	0.810	0.810
Keyimplementationpoints	23	0.550	0.478	0.512
Keymessages	31	0.850	0.548	0.667
Responsibilities	34	0.710	0.647	0.677

E1 has the highest observed Pharo final micro-F1. Figure 12 compares its label scores with the other configurations.

C Section-to-Artifact Map

The scientific evidence snapshot is commit `8a84e2cc2294f42a09914ab0185114b2abc4dc9b` of https://github.com/binhbinh22/Master-Seminar-2. Current report-branch additions include the dataset storage adapters and tests. Original model results are retained. Table 20 replaces per-section source-path blocks in the narrative.

Table 20: Primary artifacts supporting each report component.

Report component	Primary artifacts
Cover	`docs/report/report.tex` (institution, instructor, five names/IDs)
Dataset / EDA	`data/*.csv`; `data/k_fold_data/`; `data/full_training_data/`; three EDA notebooks
Dataset persistence	`src/data/storage.py`; `src/data/two_stage.py`; `tests/test_storage.py`
ML methods	`src/models/ml/features.py`; `src/models/ml/models.py`; ML staged runner
Neural methods	`configs/exp*.yaml`; `src/models/dl/`; `src/training/`
Stage 1 results	`results/training/kfold/`; `results/training/kfold_gpu0/`; `results/training/kfold_gpu1/`
Stage 2 / label counts	`results/training/final/`; `results/training/final_gpu0/`; `results/training/final_gpu1/`
Learning diagnostics	ML `training_curve.json`; neural `metrics.json` training histories
Figures / numerical checks	`report_rebuilt/scripts/`; `audit_facts.json`; `research_assets.json`
External comparisons	Primary publications in `references.bib`, especially NLBSE'23 Table III

Detailed run paths/hashes are supplied in `evidence_audit.md`, `source_inventory.csv` and `audit_facts.json`. The older manuscript's legacy classical values and incomplete neural comparison are not mixed into this staged cohort. The later PDF reports predate the compatible ML results now available. The top-four selection manifest and actual five-model final subset are documented separately rather than silently reconciled.